

CSCI 6461 | TOPIC 2

RISC vs. CISC: Instruction Set Design Principles & Trade-offs

ISA boundaries, microarchitectural costs, and quantitative trade-offs

Dr. J. Burns

Computer Systems Architecture — AArch64 Reference Platform

Topic 2: Module Roadmap

① ISA Foundations & Complexity Distribution

- The programmer-visible hardware contract and historical constraints.

② The CISC Architectural Model

- Register-memory operations, variable-length encodings, and microarchitectural costs.

③ The RISC Revolution & Principles

- Load/store discipline, fixed 32-bit width, and side-by-side assembly comparison.

④ Quantitative Performance, Energy & Modern Convergence

- Iron Law evaluation, Amdahl's Law, performance-per-watt, and modern μ op cracking.

PART 1

ISA Foundations & Historical Context

What Is an Instruction Set Architecture?

- An ISA is the **programmer-visible functional specification** of a processor.
- Defines instructions, register files, addressing modes, and architectural state.
- Sits directly at the hardware/software boundary as a rigid abstraction contract.
- **Complexity Redistribution:** ISA design redistributes complexity between software (compilers) and hardware (microarchitectures).



A well-designed ISA survives decades of microarchitectural overhauls by maintaining this strict software contract while hardware implementation evolves underneath it.

Historical Constraints Shaped Early ISAs (CISC Era)

- **Expensive, Slow Memory (1960s–1970s):** Small core memory and high DRAM latency placed a premium on minimizing binary program size.
- **Assembly-Centric Programming:** Programs were frequently hand-written; rich instructions helped express high-level constructs directly.
- **Microprogrammed ROM Control:** Complex multi-cycle instructions were cheap to implement via on-chip microcode routines.
- **The Semantic Gap:** Early designers believed machine instructions should mirror high-level statements directly.

The CISC Philosophy

Designers prioritized memory efficiency and expressive instructions to support human assembly programmers, willingly trading hardware decoding complexity for smaller binary sizes.

The CISC Architectural Model

The CISC Architectural Model

- **CISC:** *Complex Instruction Set Computer* (e.g., VAX, x86).
- **Core Characteristics:**
 - **Variable-Length Encodings:** Instructions range from 1 to 15 bytes to maximize code density.
 - **Register-Memory Operations:** Arithmetic instructions can accept memory operands directly (e.g., `add [rdi], rax`).
 - **Complex Addressing Modes:** Evaluates scaled indices and indirect pointers in hardware.
- **Microarchitectural Cost:** Requires multi-stage sequential decoders and handles highly irregular execution latencies.

The Decoding Bottleneck

Parsing variable-length instructions sequentially creates a fundamental thermal and frequency limit in modern superscalar front-ends, consuming massive transistor budgets.

CISC Execution: Register-Memory Translation

CISC Instruction Example (x86-64)

```
add [rdi + 8], rax
```

Architectural View (Software)

- 1 single instruction fetched.
- Compact variable encoding.
- Atomic arithmetic operates directly on main memory.

Hardware Decomposition (μ ops)

- 1 Compute effective address ($\text{rdi} + 8$).
- 2 Load memory contents into ALU.
- 3 Add contents of register `rax`.
- 4 Store result back to memory.

One architectural instruction decomposes into multiple hardware micro-operations, completely abstracting the pipeline timing away from the compiler.

PART 3

The RISC Revolution & Principles

The 1980s Shift: Emergence of RISC

- **Empirical Workload Findings:** Patterson & Hennessy demonstrated that compiled code utilized $< 20\%$ of available CISC instruction repertoires over 90% of the time.
- **Pipeline Hazards:** Variable-length, irregular instructions created severe pipeline stalls and hazard control logic bottlenecks.
- **Compiler Advancements:** Modern optimizing compilers handle register allocation and instruction scheduling far more effectively than monolithic hardware.

The Compiler's New Role

Instead of relying on rigid hardware to interpret complex commands, RISC shifts the burden of scheduling, register allocation, and optimization entirely to the compiler.

The RISC Design Principles

- **Fixed 32-Bit Instruction Width:** Simplifies parallel instruction fetch and multi-decoder design.
- **Strict Load/Store Architecture:** ALU operations use *only* registers; memory access is restricted to explicit `ldr/str`.
- **Large Orthogonal Register File:** 31 general-purpose registers (AArch64 `x0–x30`) maintain active variables on-chip.
- **Single-Cycle Execution Goal:** Uniform pipeline stage latencies prevent instruction bubbles and hardware stalling.

The Hardware Benefit

By simplifying the logic required to decode and route instructions, processors can redirect massive chip area into larger L1 caches, more execution units, and deeper predictive pipelines.

Code Comparison: RISC vs. CISC Memory Addition

High-level statement: $c = a + b$; (operands in memory)

AArch64 RISC Assembly

<code>ldr x1, [x0, #0]</code>	load <i>a</i>
<code>ldr x2, [x0, #8]</code>	load <i>b</i>
<code>add x3, x1, x2</code>	compute sum
<code>str x3, [x0, #16]</code>	store <i>c</i>

x86-64 CISC Assembly

<code>mov rax, [rdi]</code>	load <i>a</i>
<code>add rax, [rdi + 8]</code>	load <i>b</i> + add
<code>mov [rdi + 16], rax</code>	store <i>c</i>

RISC explicitly uses 4 fixed-width instructions (16 bytes); CISC uses 3 variable-length instructions, but the actual silicon workload executed in the hardware datapath is identical.

Quantitative Performance & Trade-offs

The Performance Equation (Iron Law)

CPU Execution Time Equation

$$T_{\text{CPU}} = IC \times CPI \times T_{\text{clk}} = \frac{IC \times CPI}{f_{\text{clk}}}$$

- **Instruction Count (IC):** Total executed instructions (influenced by ISA and compiler).
- **CPI:** Average clock cycles per instruction (influenced by pipeline and memory stalls).
- **Clock Period (T_{clk}):** Duration of one cycle (influenced by circuit depth and design).

The Optimization Tug-of-War

Enhancing one metric frequently degrades another. For instance, deepening a pipeline to increase f_{clk} often increases CPI due to more expensive branch misprediction penalties.

Worked Performance Comparison

Workload evaluation across two competing microarchitectures:

Machine A (RISC / AArch64)

- $IC = 1.4 \times 10^9$
- $CPI = 1.10$
- $f_{clk} = 3.0 \text{ GHz}$

$$T_A = \frac{1.4 \times 10^9 \times 1.10}{3.0 \times 10^9} = \mathbf{0.513 \text{ s}}$$

Machine B (CISC / x86-64)

- $IC = 1.0 \times 10^9$ (-28% IC)
- $CPI = 1.60$ (Higher CPI)
- $f_{clk} = 3.0 \text{ GHz}$

$$T_B = \frac{1.0 \times 10^9 \times 1.60}{3.0 \times 10^9} = \mathbf{0.533 \text{ s}}$$

RISC wins (Speedup $\approx 1.04\times$) because its highly predictable, lower CPI strictly compensates for executing more total operations.

Fetch Complexity: Fixed vs. Variable Width

RISC Fixed 32-Bit Decode

- Instruction k is located directly at $pc + 4k$.
- Enables simple parallel decoders (e.g., 8-wide decode).
- Low front-end power consumption and die area.

CISC Variable-Length Decode

- Lengths span 1 to 15 bytes.
- Requires sequential pre-decoders to determine instruction boundaries.
- High front-end complexity; requires decoded μop caches.

Power Implications

The computational effort of determining where one CISC instruction ends and the next begins requires massive transistor budgets, directly degrading a chip's Performance-per-Watt.

Amdahl's Law and Performance Limits

Amdahl's Law Equation

$$S_{\text{overall}} = \frac{1}{(1 - f) + \frac{f}{S_{\text{enhanced}}}}$$

- f : Fraction of execution time subjected to optimization.
- S_{enhanced} : Speedup factor achieved on that specific fraction.
- **Example** ($f = 0.60, S_{\text{enhanced}} = 10$):

$$S_{\text{overall}} = \frac{1}{0.40 + \frac{0.60}{10}} \approx 2.17\times$$

- **Takeaway:** Overall speedup is strictly capped by the non-accelerated fraction $(1 - f)$.
- This mathematically proves the **Law of Diminishing Returns** in computer architecture: optimizing the common case is always more effective than accelerating niche instructions.

Energy, Thermal Limits & Performance-per-Watt

Dynamic Power Equation

$$P = C \cdot V_{\text{dd}}^2 \cdot f + P_{\text{static}}$$

- Complex decoders increase switching capacitance (C).
- Higher frequencies (f) require higher voltages (V_{dd}).

Performance-per-Watt

$$\frac{\text{Workload Throughput}}{\text{Watts}}$$

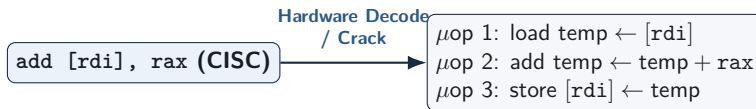
- Primary metric for mobile batteries and cloud data center racks.
- Streamlined RISC front-ends deliver high IPC at low decode energy.

The Dark Silicon Era

Because modern dense transistors cannot all be powered simultaneously without melting the chip, power efficiency is now the primary limit dictating multi-core processor performance.

Microarchitectural Convergence: Modern x86 Internals

Modern x86 processors preserve the CISC ISA externally, but execute a **RISC datapath internally**:



- The execution core is an out-of-order engine processing RISC-like micro-operations (μ ops).
- CISC-to-RISC translation occurs dynamically, incurring a front-end decode power overhead that RISC architectures natively bypass.

Review and Self-Test Questions

Topic 2: Comprehensive Summary

- **Complexity Redistribution:** ISA design distributes work between compilers and hardware decoders.
- **CISC Trade-offs:** Prioritizes code density and low instruction count via variable-length instructions and register-memory operations, at the cost of front-end decode complexity.
- **RISC Trade-offs:** Prioritizes pipeline regularity, fixed 32-bit width, and explicit load/store architecture to achieve high clock frequency and low CPI.
- **Microarchitectural Convergence:** Modern x86 processors crack CISC instructions into internal RISC μ ops, validating the core RISC execution model.
- **Evaluation Metrics:** True architectural efficiency is measured via the Iron Law ($T_{\text{CPU}} = \frac{IC \times CPI}{f_{\text{clk}}}$) and Performance-per-Watt.

Self-Test Questions: ISA Principles & Mechanics

- ① **Load/Store Boundary:** Why does a strict load/store architecture simplify pipelined execution compared to an architecture permitting memory operands directly in ALU instructions?
- ② **Parallel Decode Efficiency:** Why is an 8-wide instruction decoder significantly simpler and more power-efficient for a fixed 32-bit RISC ISA than for variable-length CISC?
- ③ **Micro-operation Translation:** When an x86 processor executes `add [rdi + 8], rax`, how does the front-end decoder functionally translate this instruction into internal μops ?

Self-Test Questions: Quantitative Analysis & Power

- ① **Iron Law Calculation:** A RISC core executes 1.5×10^9 instructions at $\text{CPI} = 1.1$ on a 3.0 GHz CPU. A CISC core executes 1.1×10^9 instructions at $\text{CPI} = 1.6$ on a 3.2 GHz CPU. Which finishes faster?
- ② **Front-End Power Cost:** Why does instruction decoding consume a measurably higher percentage of core power in modern x86 processors than in modern AArch64 cores?
- ③ **Performance-per-Watt:** Why has Performance-per-Watt superseded raw clock frequency as the primary architectural design metric in modern computing?

Looking Ahead

Next Lecture Preview

Our Next Topic: The Hardware/Software Interface & AArch64 Programmer's Model

- **Architectural State:** Defining the precise software-visible hardware contract (registers, condition flags, stack pointer, and memory space).
- **AArch64 Register File:** Examining register naming conventions (x0–x30 vs. w0–w30), the zero register (xzr), and standard ABI calling conventions.
- **Instruction Mechanics:** Exploring foundational instruction classes, memory addressing modes, and control flow translation.